

Assignment 5: Data Visualization

Joy Wu

Fall 2025

OVERVIEW

This exercise accompanies the lessons in Environmental Data Analytics on Data Visualization

Directions

1. Rename this file `<FirstLast>_A05_DataVisualization.Rmd` (replacing `<FirstLast>` with your first and last name).
2. Change “Student Name” on line 3 (above) with your name.
3. Work through the steps, **creating code and output** that fulfill each instruction.
4. Be sure your code is tidy; use line breaks to ensure your code fits in the knitted output.
5. Be sure to **answer the questions** in this assignment document.
6. When you have completed the assignment, **Knit** the text and code into a single PDF file.

Set up your session

1. Set up your session. Load the tidyverse, here & cowplot packages, and verify your home directory. Read in the NTL-LTER processed data files for nutrients and chemistry/physics for Peter and Paul Lakes (use the tidy NTL-LTER_Lake_Chemistry_Nutrients_PeterPaul_Processed.csv version in the Processed_KEY folder) and the processed data file for the Niwot Ridge litter dataset (use the NEON_NIWO_Litter_mass_trap_Processed.csv version, again from the Processed_KEY folder).
2. Make sure R is reading dates as date format; if not change the format to date.

#1

```
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
## v dplyr      1.1.4      v readr      2.1.5
## v forcats    1.0.0      v stringr   1.5.1
## v ggplot2    3.5.1      v tibble    3.2.1
## v lubridate  1.9.3      v tidyr     1.3.1
## v purrr      1.0.2
## -- Conflicts ----- tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()     masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(here)
```

```
## here() starts at /home/guest/EDE_Fall2025
```

```
library(cowplot)
```

```
##  
## Attaching package: 'cowplot'  
##  
## The following object is masked from 'package:lubridate':  
##  
## stamp
```

```
here()
```

```
## [1] "/home/guest/EDE_Fall2025"
```

```
PeterPaul.chem.nutrients <-  
  read.csv(here("Data/Processed_KEY/NTL-LTER_Lake_Chemistry_Nutrients_PeterPaul_Processed.csv"),  
           stringsAsFactors = TRUE)  
  
Niwot.Ridge.litter <- read.csv(here("Data/Processed_KEY/NEON_NIWO_Litter_mass_trap_Processed.csv"),  
                              stringsAsFactors = TRUE)  
  
#2  
PeterPaul.chem.nutrients$sampldate <- ymd(PeterPaul.chem.nutrients$sampldate)  
Niwot.Ridge.litter$collectDate <- ymd(Niwot.Ridge.litter$collectDate)  
class(PeterPaul.chem.nutrients$sampldate)
```

```
## [1] "Date"
```

```
class(Niwot.Ridge.litter$collectDate)
```

```
## [1] "Date"
```

Define your theme

3. Build a theme and set it as your default theme. Customize the look of at least two of the following:

- Plot background
- Plot title
- Axis labels
- Axis ticks/gridlines
- Legend

```
#3  
mytheme <- theme_classic(base_size = 13) +  
  theme(axis.text = element_text(color = "black"),  
        plot.background = element_rect(fill = "white", color = NA),  
        legend.position = "right")  
theme_set(mytheme)
```

Create graphs

For numbers 4-7, create ggplot graphs and adjust aesthetics to follow best practices for data visualization. Ensure your theme, color palettes, axes, and additional aesthetics are edited accordingly.

4. [NTL-LTER] Plot total phosphorus (`tp_ug`) by phosphate (`po4`), with separate aesthetics for Peter and Paul lakes. Add line(s) of best fit using the `lm` method. Adjust your axes to hide extreme values (hint: change the limits using `xlim()` and/or `ylim()`).

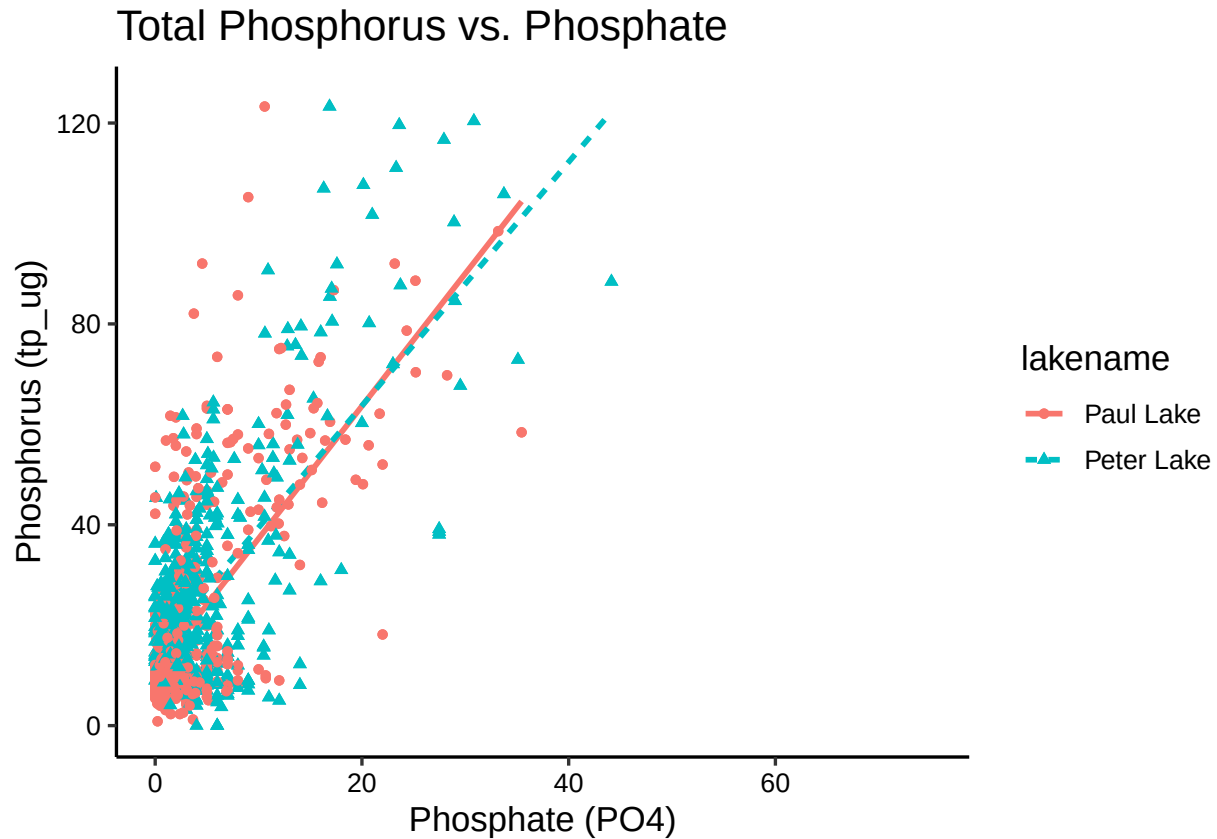
```
#4

phosphorus.by.phosphate <-
  ggplot(PeterPaul.chem.nutrients, aes(x = po4, y = tp_ug, color = lakename,
                                       shape = lakename, linetype = lakename)) +
  geom_point() +
  geom_smooth(method = "lm", se = FALSE) +
  geom_point() +
  xlim(0, 75) +
  ylim(0, 125) +
  labs(
    title = "Total Phosphorus vs. Phosphate",
    x = "Phosphate (P04)",
    y = "Phosphorus (tp_ug)"
  )
print(phosphorus.by.phosphate)

## 'geom_smooth()' using formula = 'y ~ x'

## Warning: Removed 21952 rows containing non-finite outside the scale range
## ('stat_smooth()').

## Warning: Removed 21952 rows containing missing values or values outside the scale range
## ('geom_point()').
## Removed 21952 rows containing missing values or values outside the scale range
## ('geom_point()').
```



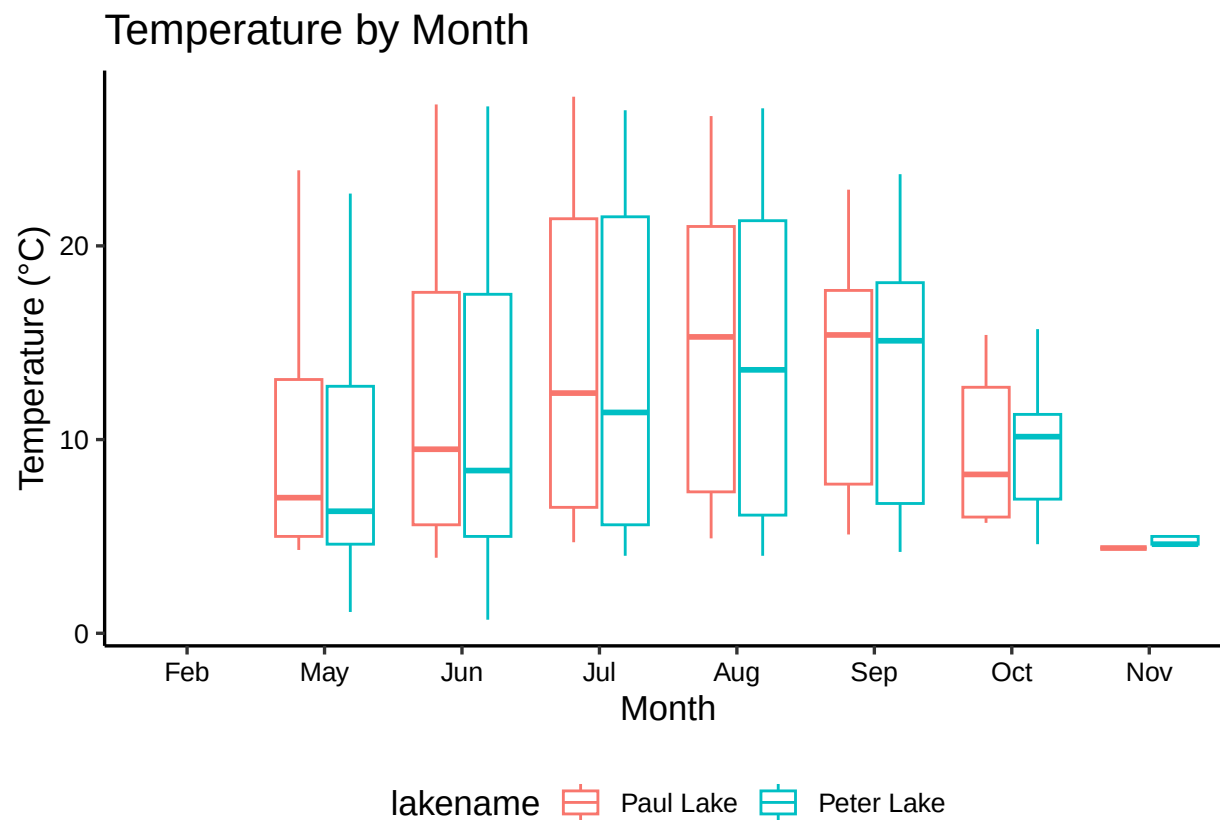
5. [NTL-LTER] Make three separate boxplots of (a) temperature, (b) TP, and (c) TN, with month as the x axis and lake as a color aesthetic. Then, create a cowplot that combines the three graphs. Make sure that only one legend is present and that graph axes are aligned.

Tips: * Recall the discussion on factors in the lab section as it may be helpful here. * Setting an axis title in your theme to `element_blank()` removes the axis title (useful when multiple, aligned plots use the same axis values) * Setting a legend's position to "none" will remove the legend from a plot. * Individual plots can have different sizes when combined using `cowplot`.

```
#5

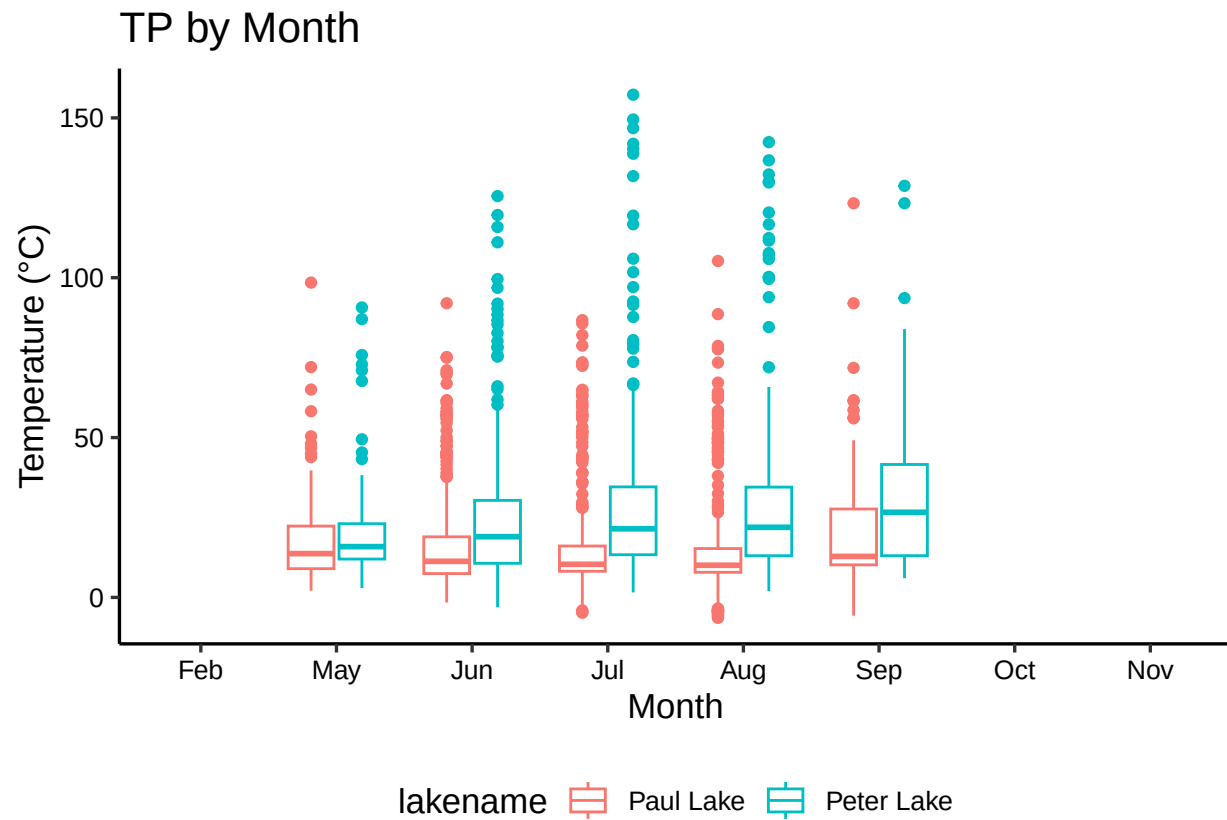
temp <- ggplot(PeterPaul.chem.nutrients,
  aes(x = factor(month, levels = 1:12, labels = month.abb),
    y = temperature_C, color = lakename)) +
  geom_boxplot() +
  labs(title = "Temperature by Month", x = "Month", y = "Temperature (°C)") +
  theme(legend.position = "bottom")
print(temp)
```

```
## Warning: Removed 3566 rows containing non-finite outside the scale range
## ('stat_boxplot()').
```



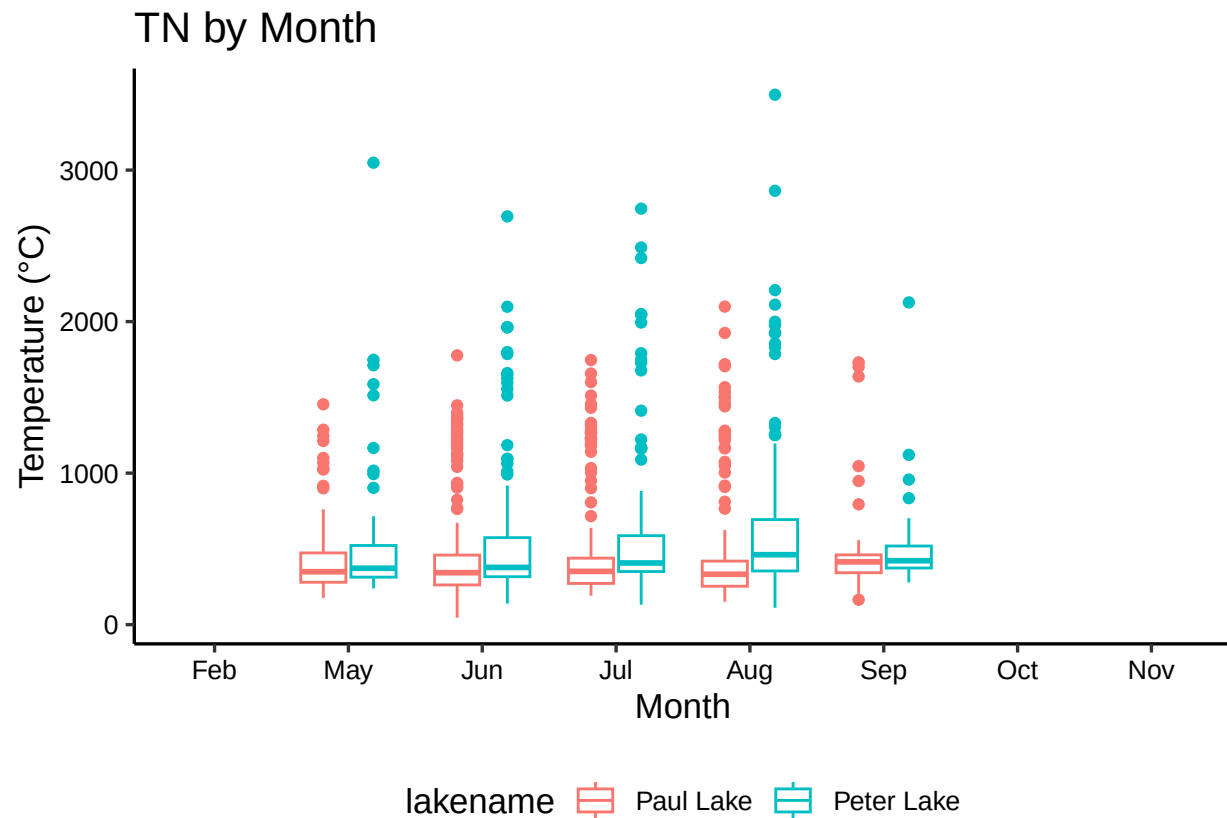
```
tp <- ggplot(PeterPaul.chem.nutrients,
             aes(x = factor(month, levels = 1:12, labels = month.abb),
                 y = tp_ug, color = lakename)) +
  geom_boxplot() +
  labs(title = "TP by Month", x = "Month", y = "Temperature (°C)") +
  theme(legend.position = "bottom")
print(tp)
```

```
## Warning: Removed 20729 rows containing non-finite outside the scale range
## ('stat_boxplot()').
```



```
tn <- ggplot(PeterPaul.chem.nutrients,
             aes(x = factor(month, levels = 1:12, labels = month.abb),
                 y = tn_ug, color = lakename)) +
  geom_boxplot() +
  labs(title = "TN by Month", x = "Month", y = "Temperature (°C)") +
  theme(legend.position = "bottom")
print(tn)
```

```
## Warning: Removed 21583 rows containing non-finite outside the scale range
## ('stat_boxplot()').
```

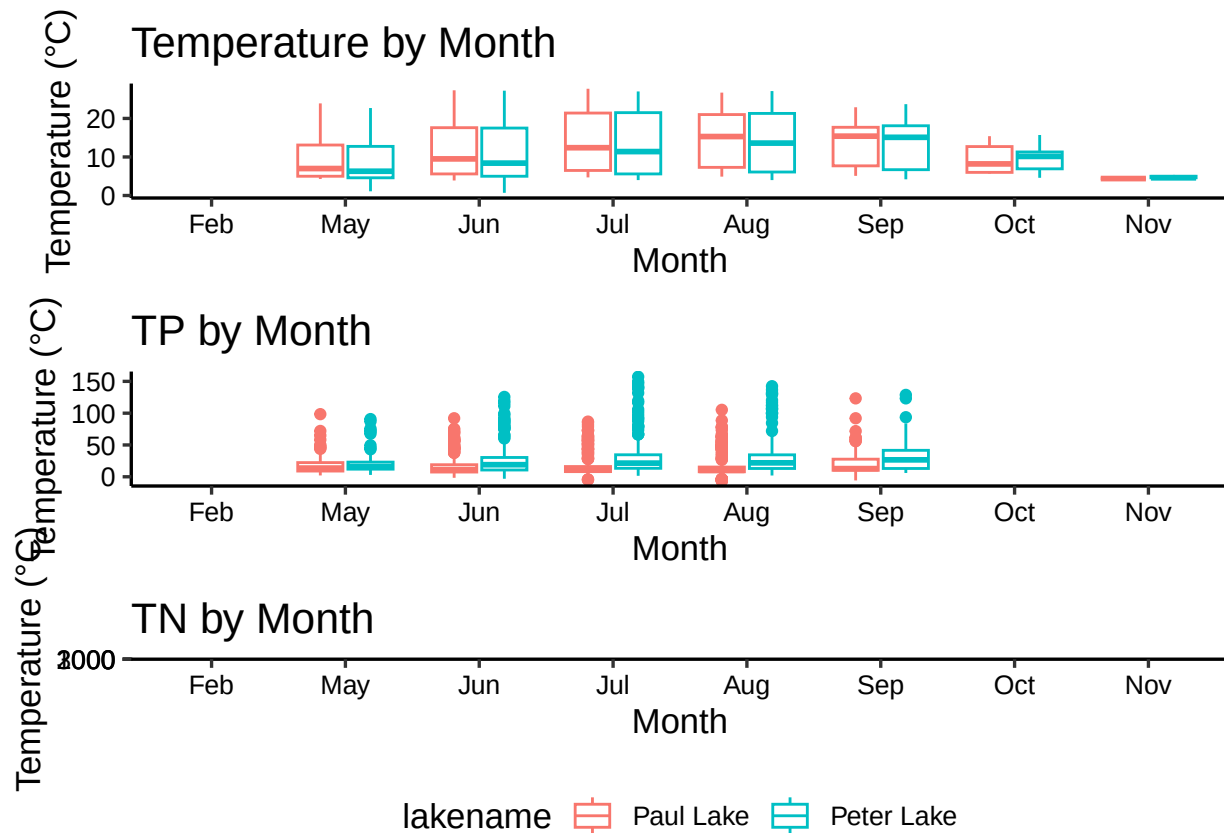


```
plot_grid(
  temp + theme(legend.position = "none"),
  tp   + theme(legend.position = "none"),
  tn   + theme(legend.position = "bottom"),
  ncol = 1,
  align = "v",
  axis = "lr",
  rel_heights = c(1, 1, 1)
)
```

```
## Warning: Removed 3566 rows containing non-finite outside the scale range
## ('stat_boxplot()').
```

```
## Warning: Removed 20729 rows containing non-finite outside the scale range
## ('stat_boxplot()').
```

```
## Warning: Removed 21583 rows containing non-finite outside the scale range
## ('stat_boxplot()').
```



Question: What do you observe about the variables of interest over seasons and between lakes?

Answer: Over seasons: Temperature increases in the summer, then decreases in the winter; TP also increases in summer times, and TN shows a more steady trend throughout the seasons. Between lakes: both lakes show a similar trend regarding temperature changes; Peter Lake generally has a higher TP and TN concentration than Paul Lake does over the months.

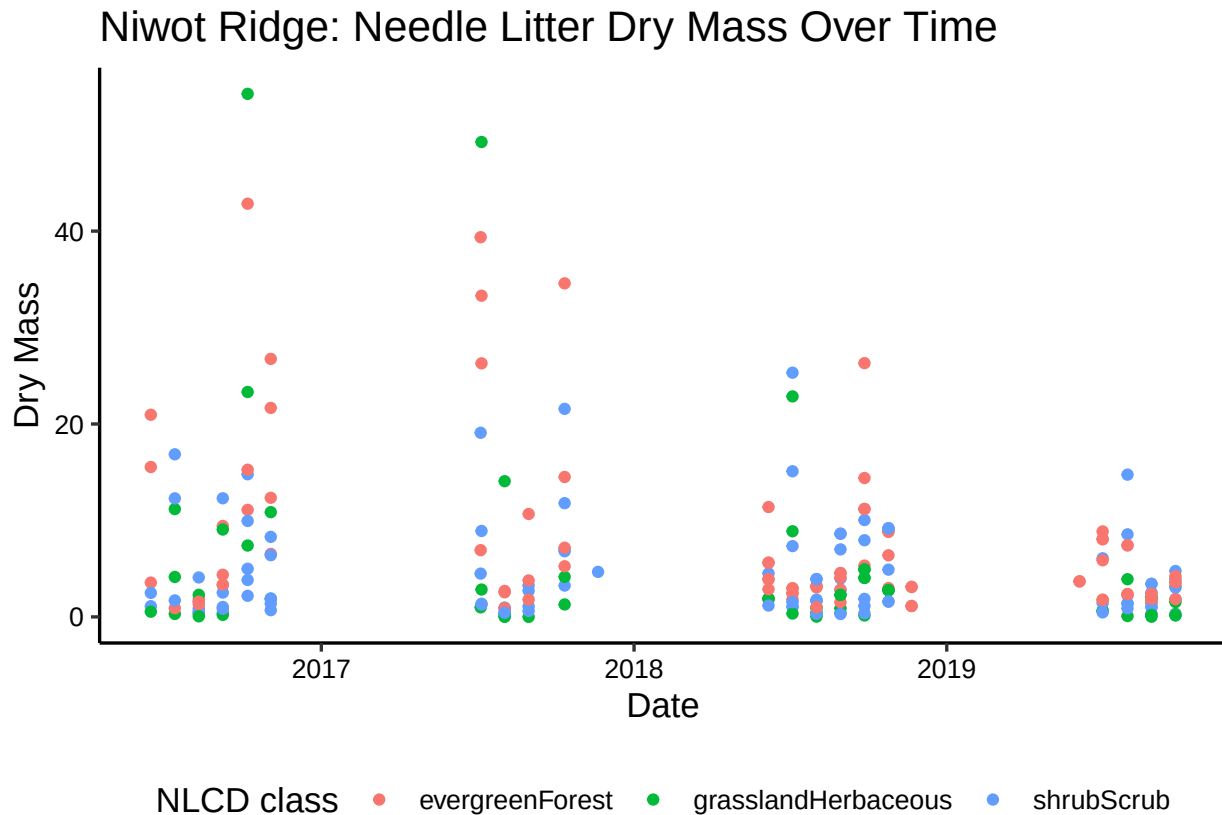
6. [Niwot Ridge] Plot a subset of the litter dataset by displaying only the “Needles” functional group. Plot the dry mass of needle litter by date and separate by NLCD class with a color aesthetic. (no need to adjust the name of each land use)
7. [Niwot Ridge] Now, plot the same plot but with NLCD classes separated into three facets rather than separated by color.

```
#6
niwot.subset <- Niwot.Ridge.litter %>%
  filter(functionalGroup == "Needles") %>%
  ggplot(
    mapping = aes(
      x=collectDate,
      y=dryMass,
      color=nlcdClass
    )
  ) +
  geom_point() +
  labs(
```

```

title = "Niwot Ridge: Needle Litter Dry Mass Over Time",
x = "Date",
y = "Dry Mass",
color = "NLCD class"
) +
theme(legend.position = "bottom")
print(niwot.subset)

```

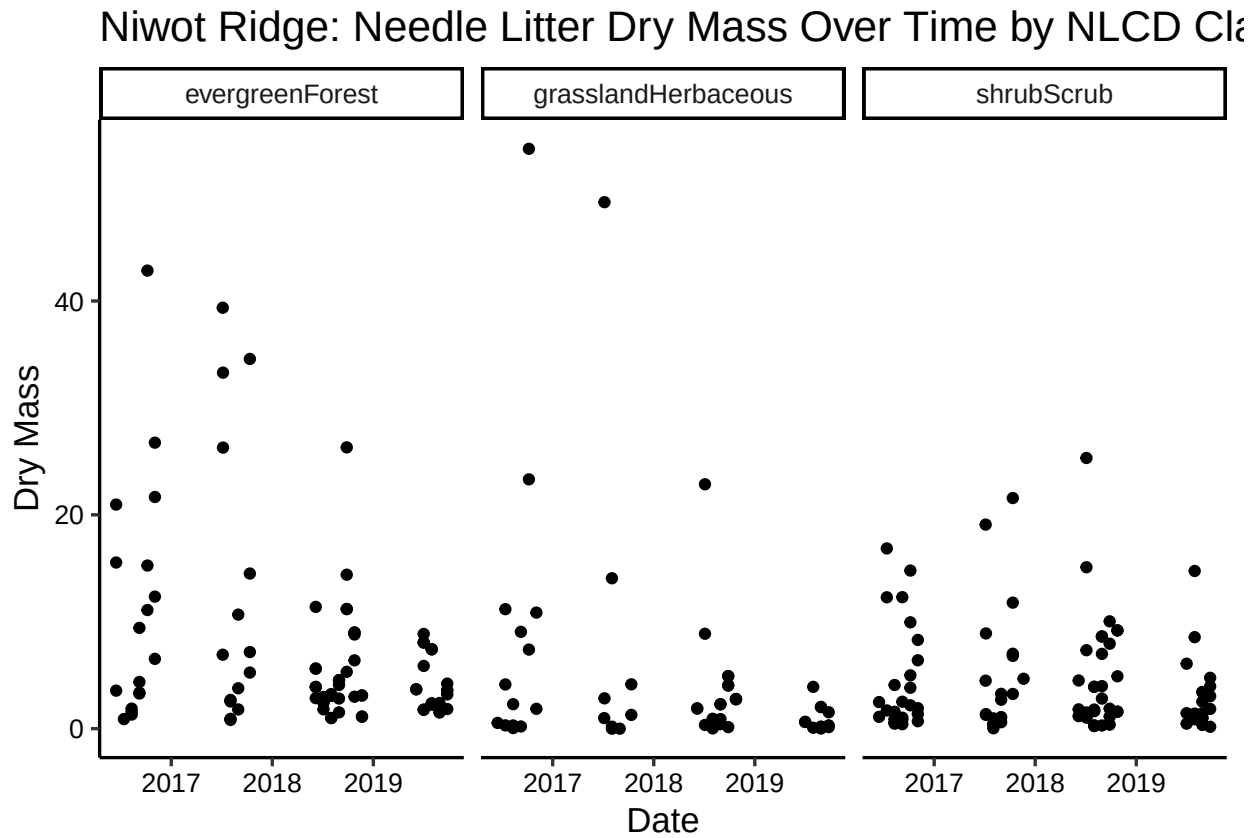


```

#7
niwot.facet <- Niwot.Ridge.litter %>%
  filter(functionalGroup == "Needles") %>%
  ggplot(
    mapping = aes(
      x=collectDate,
      y=dryMass
    )
  ) +
  geom_point() +
  facet_wrap(facets = vars(nlcdClass)) +
  labs(
    title = "Niwot Ridge: Needle Litter Dry Mass Over Time by NLCD Class",
    x = "Date",
    y = "Dry Mass"
  ) +
  theme(legend.position = "bottom")

```

```
print(niwot.facet)
```



Question: Which of these plots (6 vs. 7) do you think is more effective, and why?

Answer: I think plot 7 is more effective because it identifies patterns within each NLCD class and also allows for cross comparison between classes as they are placed next to each other.